COURSEWORK ASSIGNMENT SUBMISSION

Name: Daksh Srivastava **Roll No:** 150096725087
Subject: Software Engineering & Project Management **Day:** Day 17 — Object-Oriented Design & UML Diagrams

---

Section 12: Classroom Practical — CEMS UML Design Portfolio

Comprehensive Unified Modeling Language (UML) structural and behavioral portfolio for the College Event Management System.

1. Use Case Diagram Specification

Primary Actor	Use Case Title	Included / Extended Use Cases
Student	Browse & Filter Events	None
Student	Register for Event	Includes: `Authenticate User`, `Check Capacity`; Extends: `Join Waitlist`
Student	Cancel Registration	Includes: `Authenticate User`, `Release Seat`
Event Coordinator	Create Event Draft	Includes: `Authenticate User`
Faculty Mentor	Approve / Reject Event	Includes: `Validate Venue Availability`
Gate Volunteer	Scan Attendee QR Code	Includes: `Verify Token Signature`, `Mark Attendance`

2. Class Diagram Structural Model

- **Class `User`:** Attributes: `userId: UUID`, `name: String`, `email: String`, `role: Enum [STUDENT, COORDINATOR, FACULTY, ADMIN]`. Methods: `login()`, `logout()`, `getProfile()`.
- **Class `Student` (inherits from `User`):** Attributes: `rollNumber: String`, `department: String`. Methods: `registerForEvent(eventId)`, `cancelBooking(bookingId)`.
- **Class `Event`:** Attributes: `eventId: UUID`, `title: String`, `maxCapacity: Int`, `currentCount: Int`, `eventDate: DateTime`, `status: Enum [DRAFT, APPROVED, PUBLISHED, COMPLETED]`. Methods: `isAvailable(): Boolean`, `incrementCount()`, `decrementCount()`.
- **Class `Registration`:** Attributes: `registrationId: UUID`, `studentId: UUID`, `eventId: UUID`, `status: Enum [CONFIRMED, WAITLISTED, CANCELLED]`, `qrToken: String`, `checkedIn: Boolean`. Methods: `generateQR()`, `markCheckedIn()`.
- **Relationships:** `User` 1 <|-- `Student`; `Student` 1 --- 0..* `Registration`; `Event` 1 --- 0..* `Registration` (Composition); `Event` 1 --- 1 `FacultyMentor` (Association).

3. Sequence Diagram (Event Registration Flow)

1. `StudentUI` -> `BookingController`: `submitRegistration(studentId, eventId)`
2. `BookingController` -> `AuthService`: `validateSession(jwtToken)`
3. `AuthService` --> `BookingController`: `sessionValid: true`
4. `BookingController` -> `EventRepository`: `getEventWithLock(eventId)`
5. `EventRepository` --> `BookingController`: `eventEntity (capacity: 100, current: 85)`
6. `BookingController` -> `Event`: `isAvailable()` -> returns `true`
7. `BookingController` -> `RegistrationRepository`: `save(newRegistration)`
8. `BookingController` -> `Event`: `incrementCount()`
9. `BookingController` -> `NotificationQueue`: `publishConfirmationJob(payload)`
10. `BookingController` --> `StudentUI`: `HTTP 201 Created (registrationToken: "CEMS-8492")`

4. State Machine Diagram (Event Lifecycle)

Initial State [*] -> [DRAFT] (Trigger: Coordinator submits event draft)
[DRAFT] -> [UNDER_REVIEW] (Trigger: Coordinator requests approval)
[UNDER_REVIEW] -> [APPROVED] (Trigger: Faculty sign-off)
[UNDER_REVIEW] -> [REJECTED] (Trigger: Faculty revision required)
[APPROVED] -> [PUBLISHED] (Trigger: Scheduled release date reached)
[PUBLISHED] -> [REGISTRATION_CLOSED] (Trigger: Capacity full OR deadline reached)
[REGISTRATION_CLOSED] -> [COMPLETED] (Trigger: Event finishes)
[COMPLETED] -> Final State [*]

Section 15: Homework — Library Management System UML Suite

Object-oriented structural modeling for an automated Library Management System.

- **1. Use Case Model:** Actors: Student, Faculty, Librarian. Use Cases: Search Catalog, Checkout Book, Return Book, Pay Fine, Renew Book, Reserve Title.
- **2. Class Diagram:** `Book` (ISBN, Title, Author) 1 --- 1..* `BookCopy` (BarcodeId, Status); `Patron` (PatronId, Name, MaxLoans) 1 --- 0..* `Loan` (LoanId, IssueDate, DueDate, ReturnDate); `Loan` * --- 1 `BookCopy`; `Librarian` manages `Book` and `Loan`.
- **3. Sequence Diagram (Book Checkout with Unavailable Path):** `Student` requests book checkout. `CirculationService` queries `BookCopyRepository`. If status == `AVAILABLE`, issue loan and update status to `BORROWED`. If status == `BORROWED`, return `BookUnavailableException` and trigger alternative option prompt: "Would you like to place a Reservation Hold?"
- **4. Traceability Reflection:** The Class Diagram directly realizes SRS Requirement FR-02 (Circulation Management) by enforcing foreign key associations between `PatronId` and `BarcodeId`, guaranteeing complete traceability from requirement to implementation.